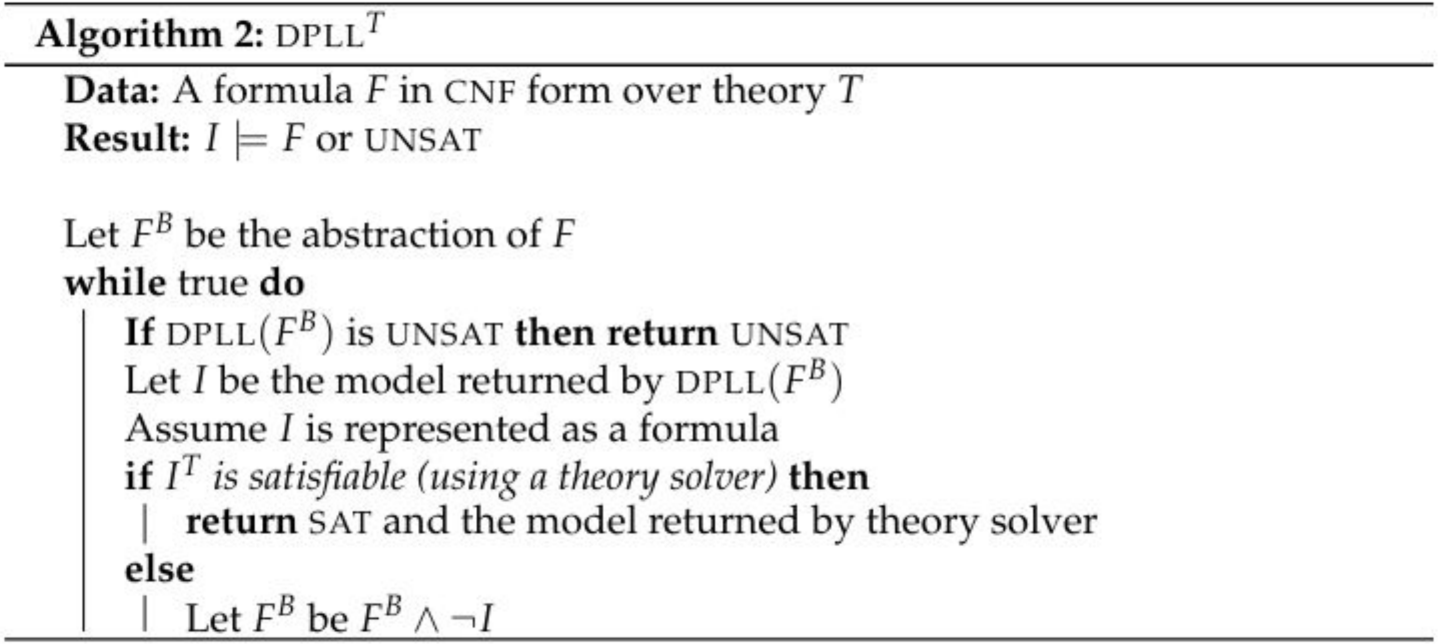
Formal Verification

Warmup: Trace DPLL^T on the following F

$(x + y \geq 4) \land (x \leq 1 \lor x \geq 3) \land (y \leq 2)$



Algorithm 2: DPLL^T

```
Data: A formula $F$ in CNF form over theory $T$
Result: $I \models F$ or UNSAT

Let $F^B$ be the abstraction of $F$
while true do
    If DPLL($F^B$) is UNSAT then return UNSAT
    Let $I$ be the model returned by DPLL($F^B$)
    Assume $I$ is represented as a formula
    if $I^T$ is satisfiable (using a theory solver) then
        return SAT and the model returned by theory solver
    else
        Let $F^B$ be $F^B \land \neg I$
```

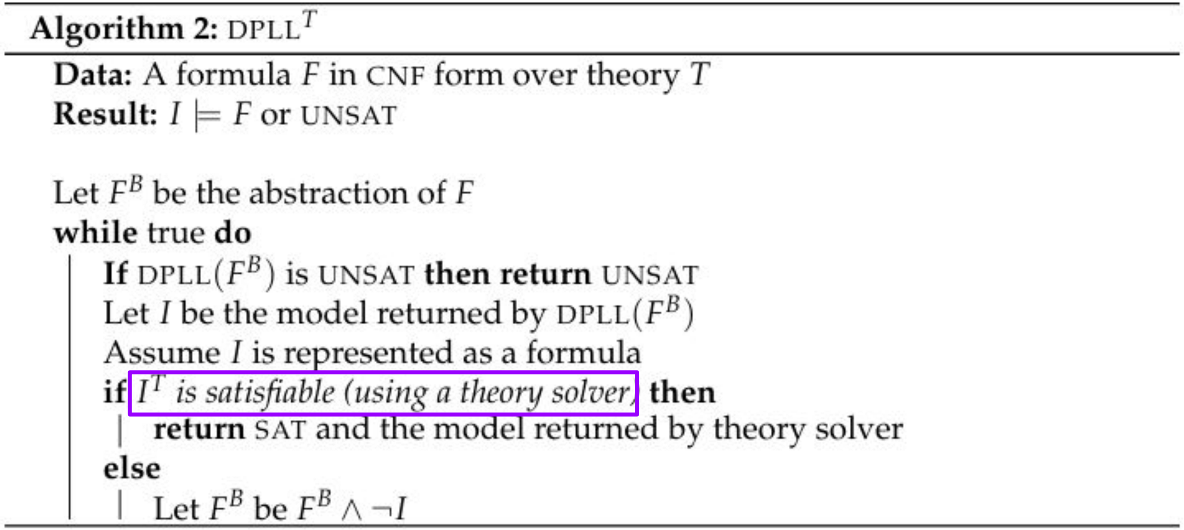
Announcements

- HW7 due Monday Nov 16th
- Thank you for coming to the talk!
- Project Checkpoint 1 due sunday November 15th

Simplex Algorithm

Simplex Algorithm

Starts with some interpretation and continues to update it every iteration



Algorithm 2: DPLL^T

```
Data: A formula $F$ in CNF form over theory $T$
Result: $I \models F$ or UNSAT

Let $F^B$ be the abstraction of $F$
while true do
    If DPLL($F^B$) is UNSAT then return UNSAT
    Let $I$ be the model returned by DPLL($F^B$)
    Assume $I$ is represented as a formula
    if $I^T$ is satisfiable (using a theory solver) then
        return SAT and the model returned by theory solver
    else
        Let $F^B$ be $F^B \wedge \neg I$
```

Simplex Form

$$\sum_i c_i \cdot x_i = 0$$

$$\ell_i \leq x_i \leq u_i$$

Translate to Simplex Form

$(x + y \geq 0) \land (-2x + y \geq 2) \land (-10x + y \geq -5)$

Take every inequality and translate it into two clauses: an equality and a bound

$(x + y \geq 0)$ ⇒

$s_1 = x + y$

$s_1 \geq 0$

$(-10x + y \geq -5)$ ⇒

$s_3 = -10x + y$

$s_3 \geq -5$

$(-2x + y \geq 2)$ ⇒

$s_2 = -2x + y$

$s_2 \geq 2$

Simplex Algorithm

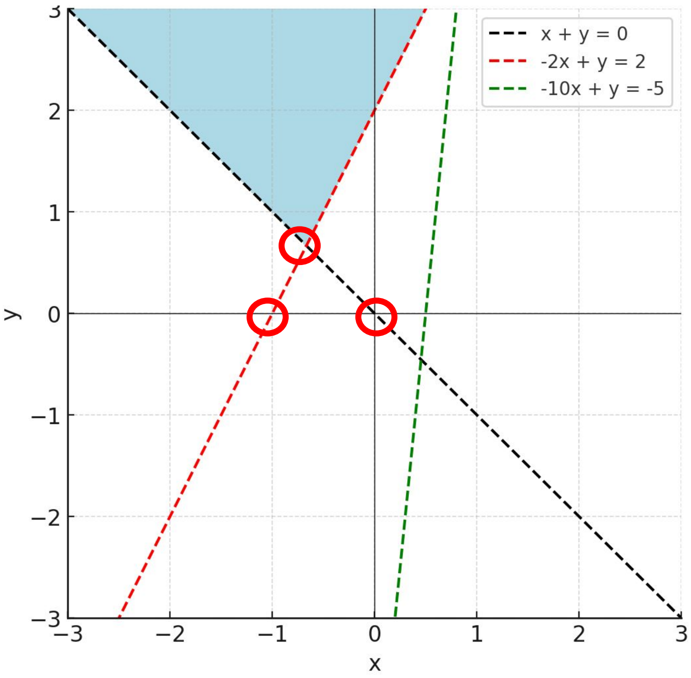
Simultaneously looks for a model and proof of unsatisfiability.

Starts with some interpretation and continues to update it every iteration

Pick a bound ($s_1 \geq 0$) that is not satisfied and modify the interpretation to satisfy it

Simplex Algorithm

$(x + y \geq 0) \land (-2x + y \geq 2) \land (-10x + y \geq -5)$



Init: $I_0 = \{x \rightarrow 0, y \rightarrow 0\}$

$I_1 = \{x \rightarrow -1, y \rightarrow 0\}$

Basic and Non-Basic Variables

Basic Variables: appear on the left side of an equality. Initially, these are the slack variables (s1, s2, s3)

Non-basic Variables: all other variables

$s_1 = x + y$

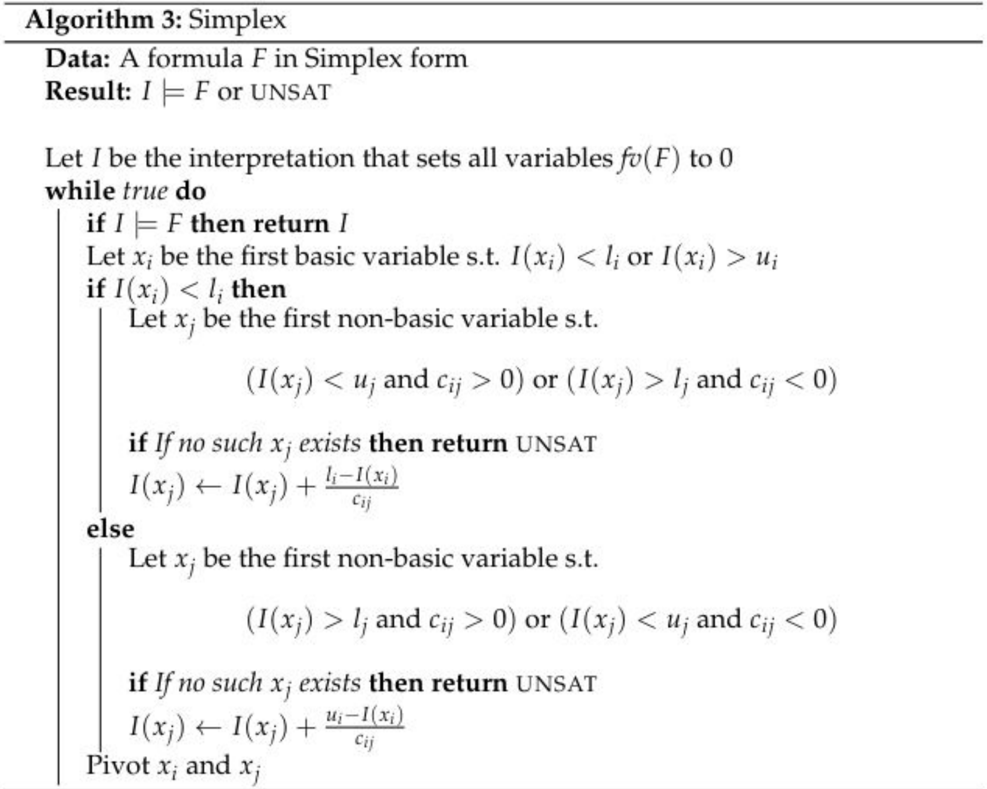
$s_1 \geq 0$

$s_2 = -2x + y$

$s_2 \geq 2$

$s_3 = -10x + y$

$s_3 \geq -5$



Algorithm 3: Simplex

```
Data: A formula $F$ in Simplex form
Result: $I \models F$ or UNSAT

Let $I$ be the interpretation that sets all variables $fv(F)$ to 0
while true do
    if $I \models F$ then return $I$
    Let $x_i$ be the first basic variable s.t. $I(x_i) < l_i$ or $I(x_i) > u_i$
    if $I(x_i) < l_i$ then
        Let $x_j$ be the first non-basic variable s.t.

            $(I(x_j) < u_j$ and $c_{ij} > 0)$ or $(I(x_j) > l_j$ and $c_{ij} < 0)$

        if If no such $x_j$ exists then return UNSAT
        $I(x_j) \leftarrow I(x_j) + \frac{l_i - I(x_i)}{c_{ij}}$
    else
        Let $x_j$ be the first non-basic variable s.t.

            $(I(x_j) > l_j$ and $c_{ij} > 0)$ or $(I(x_j) < u_j$ and $c_{ij} < 0)$

        if If no such $x_j$ exists then return UNSAT
        $I(x_j) \leftarrow I(x_j) + \frac{u_i - I(x_i)}{c_{ij}}$
    Pivot $x_i$ and $x_j$
```

Formal Verification

Bringing it all together...

How do we use SMT to prove that our program functions as expected?

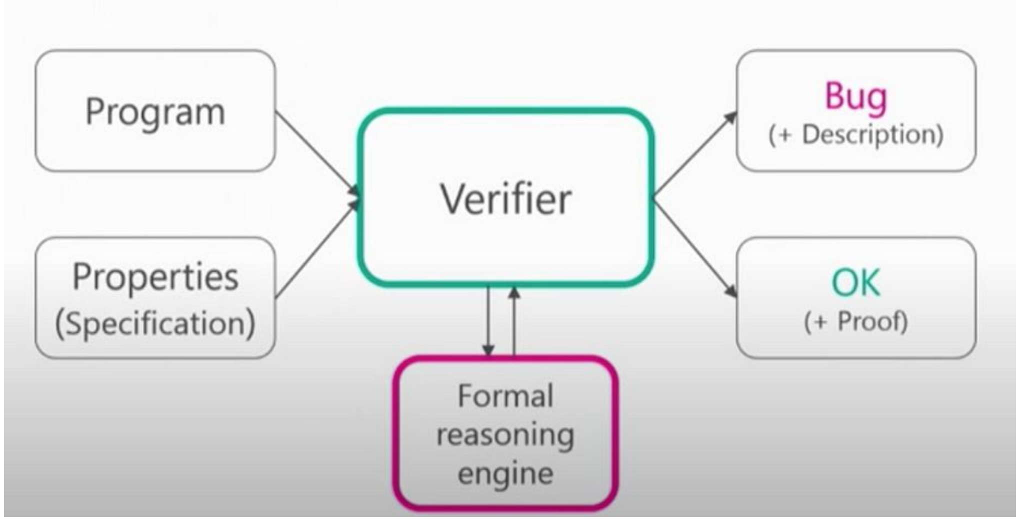
Formal Verification

```
public int addPositive(int x, int y) {

      return x + y;

}
```

Precondition:x >= 0 && y >= 0;

Assertion:

(addPos(x, y) == x + y)


Program
Properties (Specification)
Verifier
Formal reasoning engine
Bug (+ Description)
OK (+ Proof)

x >= 0 && y >= 0 && ! (addPos(x, y) == x + y)

Approaches to Formal Verification

1. Both models and the programs are encoded as a proof
2. Tools take as input a program in a particular language with an **annotation language.** Automatically produces a SMT formula which is fed to a verifier

Specifications

What Are Specifications?

Specifications describe what a program should do

A **Precondition** is a condition that must be true before method execution, and a **Postcondition** is a condition that must be true after method execution.

A **Specification** is a contract between a method and its caller, consisting of a precondition and a postcondition.

JML

JML = Java Modeling Language

Used to specify contracts for Java methods and classes

Annotations look like comments but express **preconditions**, **postconditions**, and **invariants**

Basic Syntax

//@ requires <precondition>;

//@ ensures <postcondition>;

requires: What must be true before method executes

ensures: What will be true after method completes, if preconditions were metr

The `\result` Keyword

`//@ ensures \result == x + y;`

`\result` refers to the value returned by the method

Only used in `ensures` clauses

Example

```
//@ requires x >= 0 && y >= 0;
//@ ensures \result == x + y;
public int addPositive(int x, int y) {
    return x + y;
}
```

Using Logical Operators

You can use && and || in your pre and post conditions

```
//@ requires x > 0 && y > 0;

//@ ensures \result == x * y || \result == 0;
```

Example 2

```
//@ requires n >= 0;

//@ ensures (\result == 1 && n == 0) || \result == n * factorial(n - 1);

public int factorial(int n) {

    if (n == 0) return 1;

    return n * factorial(n - 1);

}
```

Conditional Logic

JML supports conditional expressions using `==>` **(implies)** and `?:` **(ternary-style)**

//@ ensures x > 0 ==> \result > 0;

“If `x > 0` is true, then `\result > 0` must be true.”

//@ ensures (x >= 0) ? \result == x : \result == -x;

“If `x >= 0`, then result is `x`, otherwise it's `-x`.”

Example:

```
//@ ensures (x >= 0) ==> \result == x;

//@ ensures (x < 0) ==> \result == -x;

public int absoluteValue(int x) {

    return (x >= 0) ? x : -x;

}
```

OpenJML

OpenJML translates JML specs into boolean constraints and passes them to an SMT solver

OpenJML

```
public class MathUtils {
    //@ requires x >= 0;
    //@ ensures \result == x + 1;
    public static int increment(int x) {
        return x + 1;
    }
}
```

`(x >= 0) => (result == x + 1)`

`(x >= 0) => ((x + 1) == x + 1)`

Feed the negation to Z3

$$(x \geq 0) \land ((x + 1) \neq x + 1)$$

UNSAT

Approaches to Formal Verification

1. Program is part of the proof
    1. Offer a higher level of assurance
    2. Annotations often end up repeating much of the program
2. Annotations on normal languages
    1. Easier to adopt

Summary

- Boolean Satisfiability
    - Path conditions can be modelled as SAT formulae
    - Solved with DPLL

- If our formula contains non-boolean variables, we need to query a “theory solver”
    - Simplex for LRA

- Specifications
    - Preconditions, postconditions, ...